

A04-Prompt-Engineering-Standards

1 Prompt Engineering Standards

Document Version: 1.0

Generated: December 4, 2025

Standard: Prompt Design Patterns and Best Practices

Status: Emerging Standard

1.1 Overview

Prompt engineering is the practice of designing inputs to LLMs to achieve desired outputs. Standardized patterns enable:

- **Consistency:** Reproducible results
- **Quality:** Better answer quality
- **Efficiency:** Fewer tokens, lower cost
- **Reliability:** Reduced hallucinations
- **Explainability:** Clear reasoning

1.1.1 Core Principles

1. **Clarity:** Explicit instructions
 2. **Structure:** Formatted inputs
 3. **Examples:** Few-shot learning
 4. **Constraints:** Output specification
 5. **Context:** Relevant information
 6. **Role-Playing:** Clear persona
-

1.2 Authoritative References

- **OpenAI Prompt Guide:** <https://platform.openai.com/docs/guides/gpt-best-practices>
 - **Anthropic Claude:** <https://docs.anthropic.com/en/docs/build-a-bot>
 - **Prompt Engineering Guide:** <https://www.promptingguide.ai>
 - **ReAct Framework:** <https://arxiv.org/abs/2210.03629>
-

1.3 Format Structure

1.3.1 Basic Prompt Format

Standard template:

```
[ROLE]
You are a {role}. Your task is to {task}.

[CONTEXT]
{background_information}

[INSTRUCTIONS]
1. {instruction_1}
2. {instruction_2}
3. {instruction_3}

[OUTPUT_FORMAT]
Provide your response in the following format:
- Key 1: {format}
- Key 2: {format}

[CONSTRAINTS]
- {constraint_1}
- {constraint_2}

[EXAMPLES]
Example 1:
Input: {example_input}
Output: {example_output}
```

1.3.2 JSON Format (Structured)

Structured prompt with JSON:

```
{
  "system_prompt": "You are a helpful assistant specialized in technical documentation.",
  "user_task": "Summarize the following article",
  "input": {
    "content": "Article text here..."
  },
  "output_schema": {
    "summary": "2-3 sentence summary",
    "key_points": ["point1", "point2", "point3"],
    "sentiment": "positive|neutral|negative"
  },
  "constraints": {
    "max_tokens": 500,
    "language": "English",
    "style": "professional"
  }
}
```

1.3.3 XML Format (Advanced)

XML-based structured prompts:

```
<?xml version="1.0" encoding="UTF-8"?>
<prompt>
  <system_prompt>
    You are an expert technical writer specializing in API documentation.
  </system_prompt>

  <task>
    Analyze the provided code and generate API documentation.
  </task>

  <context>
    <language>Python</language>
    <framework>FastAPI</framework>
    <audience>Junior developers</audience>
  </context>

  <instructions>
    <instruction priority="high">
      Explain each parameter with its type and purpose
    </instruction>
    <instruction priority="high">
      Include example usage
    </instruction>
    <instruction priority="medium">
      Note any error cases
    </instruction>
  </instructions>

  <output_format>
    <format type="markdown">
      # Function Name
      ## Parameters
      ## Returns
      ## Example
    </format>
  </output_format>

  <examples>
    <example id="1">
      <input>def get_user(id: int) -> dict: ...</input>
      <output># get_user
      Retrieves user information by ID...
    </example>
  </examples>
</prompt>
```

1.4 Procedural Use-Cases

1.4.1 Use-Case 1: Few-Shot Prompting

Principle: Examples improve model performance

```

class FewShotPrompt:
    def __init__(self, task: str):
        self.task = task
        self.examples = []

    def add_example(self, input_text: str, output_text: str):
        """Add example for in-context Learning"""
        self.examples.append({
            "input": input_text,
            "output": output_text
        })

    def build_prompt(self, query: str) -> str:
        """Build few-shot prompt"""

        prompt = f"Task: {self.task}\n\n"

        # Add examples
        for i, example in enumerate(self.examples, 1):
            prompt += f"Example {i}:\n"
            prompt += f"Input: {example['input']}\n"
            prompt += f"Output: {example['output']}\n\n"

        # Add query
        prompt += f"Now, solve this:\n"
        prompt += f"Input: {query}\n"
        prompt += f"Output:"

        return prompt

# Usage
prompt_builder = FewShotPrompt("Classify sentiment")

prompt_builder.add_example(
    "Great product, love it!",
    "Sentiment: Positive"
)

prompt_builder.add_example(
    "Terrible experience, never again.",
    "Sentiment: Negative"
)

full_prompt = prompt_builder.build_prompt("This movie was amazing")
print(full_prompt)

```

1.4.2 Use-Case 2: Chain-of-Thought (CoT)

Principle: Explicit reasoning improves accuracy

```
def chain_of_thought_prompt(problem: str) -> str:
    """Generate CoT prompt for complex reasoning"""

    return f"""Solve the following problem step by step.

Problem: {problem}

Think through this carefully:
1. First, understand what is being asked
2. Identify the key information needed
3. Plan your approach
4. Work through the solution step by step
5. Verify your answer

Your reasoning (step by step):"""

# Example
problem = "If a train travels 120 miles in 2 hours, what is its average speed?"
prompt = chain_of_thought_prompt(problem)

# LLM would produce:
# 1. We need to find average speed
# 2. Key info: distance = 120 miles, time = 2 hours
# 3. Formula: speed = distance / time
# 4. Calculation: 120 / 2 = 60 miles per hour
# 5. This makes sense for a train's speed
```

Real Implementation (Python):

```
import anthropic

def solve_with_cot(problem: str) -> str:
    """Use Claude with chain-of-thought"""

    client = anthropic.Anthropic()

    message = client.messages.create(
        model="claude-3-sonnet-20240229",
        max_tokens=1024,
        messages=[
            {
                "role": "user",
                "content": f"""Solve this problem step by step:

{problem}

Think through this carefully:
1. Understand what's asked
2. Identify key information
3. Plan your approach
4. Work through the solution
5. Verify the answer"""
            }
        ]
    )

    return message.content[0].text
```

1.4.3 Use-Case 3: ReAct Pattern (Reasoning + Acting)

Principle: Combine reasoning with tool use

```
def react_prompt_template(problem: str) -> str:
    """ReAct pattern: Thought -> Action -> Observation"""
```

```
return f"""You are an AI assistant that can reason and take actions.
```

```
Problem: {problem}
```

```
Use the following format:
```

```
Thought: <your reasoning about what to do>
```

```
Action: <tool to use from [Search, Calculate, Lookup]>
```

```
Observation: <result of action>
```

```
... (repeat as needed)
```

```
Final Answer: <solution>
```

```
Start:
```

```
Thought: ""
```

```
# Full conversation example
```

```
react_conversation = ""
```

```
Thought: I need to find information about Python's release date
```

```
Action: Search(Python release date)
```

```
Observation: Python was first released in 1991
```

```
Thought: I should also find when Python 3 was released
```

```
Action: Search(Python 3 release date)
```

```
Observation: Python 3.0 was released on December 3, 2008
```

```
Thought: Now I have enough information to answer
```

```
Final Answer: Python was first released in 1991, with Python 3 released in 2008
```

```
""
```

```
# Implementation with tool calling
```

```
def react_agent(problem: str, tools: dict):
```

```
    """Execute ReAct pattern"""
```

```
    client = anthropic.Anthropic()
```

```
    tools_definitions = [
```

```
        {
            "name": "Search",
            "description": "Search for information",
            "input_schema": {
                "type": "object",
                "properties": {
                    "query": {"type": "string"}
                }
            }
        },
```

```
        {
            "name": "Calculate",
            "description": "Perform calculations",
            "input_schema": {
                "type": "object",
                "properties": {
                    "expression": {"type": "string"}
                }
            }
        }
    ]
```

```
    messages = [
        {
            "role": "user",
            "content": react_prompt_template(problem)
        }
    ]
```

```
# Agentic Loop
```

```
while True:
```

```
    response = client.messages.create(
        model="claude-3-sonnet-20240229",
        max_tokens=1024,
        tools=tools_definitions,
        messages=messages
```

```
    )
```

```

# Check if done
if response.stop_reason == "end_turn":
    return response.content[-1].text

# Process tool calls
for content in response.content:
    if content.type == "tool_use":
        tool_result = tools[content.name](content.input)

# Add to conversation
messages.append({"role": "assistant", "content": response.content})
messages.append({
    "role": "user",
    "content": f"Observation: {tool_result}"
})
break

```

1.5 Examples

1.5.1 Example 1: System vs User Prompts

Good separation:

```

system_prompt = """You are a Python expert who:
- Provides clear, well-commented code
- Follows PEP 8 style guidelines
- Explains complex concepts simply
- Warns about edge cases"""

user_prompt = """Write a function to calculate the factorial of a number.
Requirements:
- Handle edge cases (negative numbers, zero)
- Include type hints
- Add docstring"""

# Claude API call
response = client.messages.create(
    model="claude-3-sonnet-20240229",
    max_tokens=1024,
    system=system_prompt,
    messages=[
        {"role": "user", "content": user_prompt}
    ]
)

```

1.5.2 Example 2: Structured Output

JSON-based output format:

```

def extract_with_structure(text: str) -> dict:
    """Extract structured information from text"""

    client = anthropic.Anthropic()

    response = client.messages.create(
        model="claude-3-sonnet-20240229",
        max_tokens=1024,
        messages=[
            {
                "role": "user",
                "content": f"""Extract information from this text and return as JSON:

Text: {text}

Return a JSON object with this structure:
{{
  "main_topic": "string",
  "key_concepts": ["array", "of", "concepts"],
  "sentiment": "positive|negative|neutral",
  "confidence": 0.0-1.0
}}

JSON Response: """
            }
        ]
    )

    import json
    return json.loads(response.content[0].text)

```

1.5.3 Example 3: Role-Based Prompting


```
class RoleBasedPrompt:
    """Generate prompts with different roles"""

    ROLES = {
        "teacher": "You are an experienced teacher explaining concepts",
        "developer": "You are a senior software engineer with 20 years experience",
        "writer": "You are a creative writer specializing in fiction",
        "analyst": "You are a business analyst focused on ROI"
    }

    @staticmethod
    def generate(role: str, task: str, context: str = "") -> str:
        """Generate role-based prompt"""

        role_description = RoleBasedPrompt.ROLES.get(role, "")

        prompt = f"""{role_description}

Your task: {task}

{f"Context: {context}" if context else ""}

Please provide a thorough response: """

        return prompt

# Usage
prompt = RoleBasedPrompt.generate(
    "teacher",
    "Explain machine learning to beginners",
    "Focus on practical applications"
)
print(prompt)
```

1.6 Best Practices

Pattern	Use When	Example
Zero-Shot	Task is clear, model is capable	“Translate to Spanish: Hello”
Few-Shot	Task needs examples	Classification with examples
Chain-of-Thought	Complex reasoning required	Math problems, logic
ReAct	Tool use needed	Information retrieval, calculations
Role-Play	Specific persona needed	Expert opinions, creative writing

1.7 Tools & Ecosystem

Tool	Purpose	URL
OpenAI Playground	Test prompts	https://platform.openai.com/playground
Anthropic Console	Claude testing	https://console.anthropic.com
PromptBase	Marketplace	https://promptbase.com
LangChain	Prompt templates	https://docs.langchain.com
DSPy	Optimization	https://github.com/stanfordnlp/dspy